

Here's a **complete, structured guide** to Java 8+ **Stream API methods**, categorized by whether they **accept lambda expressions (functional interfaces)** or **do NOT accept lambdas**, along with **clear Java code examples**.

❑ 1. INTERMEDIATE OPERATIONS

✔ A. Intermediate Methods that ACCEPT Lambda Expressions

These methods require **functional interfaces** like Predicate, Function, Consumer, etc.

Method	Functional Interface	Lambda Example
filter()	Predicate	<code>x -> x > 10</code>
map()	Function	<code>x -> x * 2</code>
flatMap()	Function	<code>x -> x.stream()</code>
mapToInt()	ToIntFunction	<code>x -> x.length()</code>
mapToLong()	ToLongFunction	<code>x -> x</code>
mapToDouble()	ToDoubleFunction	<code>x -> x * 1.0</code>
flatMapToInt()	Function	<code>x -> IntStream.of(...)</code>
flatMapToLong()	Function	same
flatMapToDouble()	Function	same
peek()	Consumer	<code>x -> System.out.println(x)</code>
sorted(Comparator)	Comparator	<code>(a,b) -> a - b</code>
takeWhile() (Java 9)	Predicate	<code>x -> x < 100</code>
dropWhile() (Java 9)	Predicate	<code>x -> x < 100</code>

✓ Example

```
List<Integer> list = List.of(10, 20, 30, 40);
```

```
list.stream()
    .filter(x -> x > 20)
    .map(x -> x * 2)
    .peek(x -> System.out.println("Processed: " + x))
    .sorted((a, b) -> b - a)
    .forEach(System.out::println);
```

✗ B. Intermediate Methods that DO NOT ACCEPT Lambda

These methods either:

- take **no arguments**, or
- take **non-functional arguments**

Method	Reason
distinct()	No parameter
sorted()	Uses natural ordering
limit()	Takes long
skip()	Takes long
parallel()	No parameter
sequential()	No parameter
unordered()	No parameter
onClose()	Actually takes lambda (Consumer) → special case

✗ Example

```
list.stream()
    .distinct()
    .limit(3)
    .skip(1)
    .sorted()
    .forEach(System.out::println);
```

❑ 2. TERMINAL OPERATIONS

✔ A. Terminal Methods that ACCEPT Lambda Expressions

Method	Functional Interface	Example
forEach()	Consumer	x -> print(x)
forEachOrdered()	Consumer	Same
reduce()	BinaryOperator	(a,b) -> a + b
collect()	Collector / Supplier/Accumulator	Custom
anyMatch()	Predicate	x -> x > 10
allMatch()	Predicate	x -> x > 10
noneMatch()	Predicate	x -> x > 10
findFirst()	No lambda	
findAny()	No lambda	
min()	Comparator	(a,b)->a-b
max()	Comparator	(a,b)->a-b
toArray(IntFunction)	IntFunction	size -> new int[size]

✔ Example

```
int sum = List.of(1,2,3,4)
    .stream()
    .filter(x -> x % 2 == 0)
    .reduce(0, (a, b) -> a + b);
```

```
System.out.println(sum);
```

✗ B. Terminal Methods that DO NOT ACCEPT Lambda

Method	Reason
count()	No argument
toArray()	No argument

findFirst()	No argument
findAny()	No argument
iterator()	No argument
splititerator()	No argument

✖ Example

```
long count = list.stream()
    .filter(x -> x > 10)
    .count();
```

```
System.out.println(count);
```

❑ 3. SPECIAL CASES (IMPORTANT FOR INTERVIEWS)

❑ Method Overloads (Lambda Optional)

Method	Lambda Optional?	Explanation
sorted()	Yes	sorted() vs sorted(Comparator)
reduce()	Yes	multiple overloads
collect()	Yes	predefined collectors vs custom
toArray()	Yes	default vs generator

❑ 4. REAL-TIME USE CASE

Scenario: Process Employee Data

```
class Employee {
    String name;
    int salary;

    Employee(String name, int salary) {
        this.name = name;
        this.salary = salary;
    }
}
```

```
List<Employee> employees = List.of(
```

```
new Employee("A", 50000),
new Employee("B", 70000),
new Employee("C", 60000)
);

// Get names of employees with salary > 55000
List<String> result = employees.stream()
    .filter(e -> e.salary > 55000)
    .map(e -> e.name)
    .sorted()
    .collect(Collectors.toList());

System.out.println(result);
```

❑ 5. FINAL SUMMARY (INTERVIEW READY)

✔ Methods USING Lambda

- filter, map, flatMap
- peek
- sorted(Comparator)
- reduce
- collect
- forEach
- anyMatch, allMatch, noneMatch
- min, max

✗ Methods NOT USING Lambda

- distinct, limit, skip
 - count, findFirst, findAny
 - iterator, spliterator
-

❑ Pro Tip (Very Important)

- ❑ If a method requires a **Functional Interface**, you can use a **Lambda Expression**
 - ❑ If it takes **primitive / no argument**, you **cannot use Lambda**
-

If you want next:

- ✔ 100+ Stream API interview questions (FAANG level)
- ✔ Internal working of Stream pipeline (deep dive)
- ✔ Parallel Stream internals (ForkJoinPool)